

Study Report: Python Programming - 3 Manuscripts (Crash Course, Coding with Python, Data Science)

Written by Gagan Pasupuleti

Family:	Data Science
Level:	Beginner to Intermediate
Chapters:	9
Source:	PYTHON PROGRAMMING - 3 MANUSCRIPTS CRASH COURSE CODING WITH PYTHON DATA SCIENCE.epub

Summary

This report covers a three-in-one collection that moves from a crash course, through practical coding, into data science. It teaches why to learn Python, variables and operators, simple data types, conditional and control flow statements, functions, object-oriented programming, working with files, coding fundamentals, and Python libraries, finishing with getting real programs working. It is a broad path from beginner coding to data-focused Python.

Chapters

Chapter 1: What Is Python and Why Learn It

Chapter focus: What Is Python and Why Learn It

Python is a general-purpose programming language known for readable syntax and wide use in web development, automation, data analysis, and machine learning. Unlike many languages that use braces and semicolons, Python uses indentation to show structure, which helps beginners see how code is organized. You write instructions in a .py file or run them line by line in the interactive shell. Each instruction tells the computer to do something: store a value, print text, make a decision, or repeat a task.

Code Reference:

```
print("Hello, Python!")  
print(2 + 3)
```

What it shows: The first line prints text. The second shows Python can also calculate numbers.

Topic guide

What it actually is

Python is a high-level, interpreted language: you write human-readable code and the Python interpreter runs it without a separate compile step. It comes with a large standard library and thousands of third-party

packages.

When to use it

Use Python when you want to build something quickly, automate repetitive tasks, analyze data, create a web backend, or learn programming for the first time.

Where to use it

Used at Google, Netflix, NASA, and in schools worldwide. Common in data science teams, DevOps automation, scripting, and beginner coding courses.

Real use example

A small business owner writes a 10-line Python script that renames 200 invoice PDFs by date every month, saving an hour of manual work.

Chapter 2: Variables and Operators

Chapter focus: Variables and Operators

Operators are symbols that perform actions on values: + - * / for math, == != < > for comparisons, and and or not for logic. Assignment operators like += update a variable in place (count += 1 means count = count + 1). Understanding precedence avoids bugs - parentheses make order explicit.

Code Reference:

```
a, b = 10, 3
print(a + b, a // b, a % b)    # 13 3 1
print(a > b and b > 0)        # True
```

What it shows: // is integer division; % is remainder; and combines two True/False tests.

Topic guide

What it actually is

Operators are symbols that tell Python to compute, compare, or combine values. Expressions like price * 1.18 produce new values from existing ones.

When to use it

Use operators in every calculation, comparison, and logical test - totals, averages, valid ranges, and combining yes/no checks.

Where to use it

Calculators, grading logic, game scoring, filtering data, and updating counters in loops.

Real use example

A shopping cart uses total += price * qty inside a loop to accumulate the bill instead of rewriting total = total + ... every time.

Chapter 3: Simple Data Types

Chapter focus: Simple Data Types

A variable is a name that points to a value stored in memory. You create one with the assignment operator (=): the name goes on the left, the value on the right. Python figures out the type automatically - you do not declare int or str first. Common types are integers (whole numbers), floats (decimals), strings (text in quotes), and booleans (True/False). You can change a variable later by assigning a new value. Clear names like `user_age` or `total_price` make code easier to read than single letters.

Code Reference:

```
count = 5          # integer
label = "box"      # string
price = 2.5        # float
is_open = True     # boolean
print(type(count)) # <class 'int'>
```

What it shows: Each variable stores one value. `type()` tells you what kind of data it holds.

Topic guide

What it actually is

Variables are labeled containers for data. The label (name) lets you reuse and update values throughout a program without repeating the actual number or text.

When to use it

Use variables whenever a value might change, will be reused later, or needs a meaningful name - scores, user names, prices, flags, counters.

Where to use it

Every Python program: games (player score), web apps (username), data scripts (file path), calculators (operands), and loops (counters).

Real use example

An online shop stores `item_price = 499`, `quantity = 2`, and computes `total = item_price * quantity` so the checkout page shows Rs 998 without hard-coding numbers.

Chapter 4: Conditional and Control Flow Statements

Chapter focus: Conditional and Control Flow Statements

Conditional statements let a program choose different actions based on whether a condition is True or False. Start with `if`, add `elif` for extra tests, and `else` for the fallback. Only one branch runs - the first condition that is true. Conditions use comparison operators (`==`, `!=`, `<`, `>`) and logical operators (`and`, `or`, `not`). Indentation shows which lines belong to each branch.

Code Reference:

```
score = 76
if score >= 90:
    grade = "A"
elif score >= 60:
    grade = "Pass"
```

```
else:
    grade = "Fail"
print(grade) # Pass
```

What it shows: Each condition is checked top to bottom; the first true branch sets grade.

Topic guide

What it actually is

A conditional is a decision gate in code. The program evaluates a boolean expression and runs only the matching block of statements.

When to use it

Use conditionals whenever the program should behave differently based on data: age checks, valid input, empty lists, file exists, user role.

Where to use it

Login systems, grading, game win/lose, error handling paths, menu choices, and filtering data.

Real use example

An ATM checks if balance $\geq$ amount: if true it withdraws; elif balance > 0 it shows 'insufficient funds'; else it shows 'zero balance'.

Chapter 5: Working with Functions

Chapter focus: Working with Functions

Keep functions small and named after what they do. Document tricky ones with a one-line docstring. Return early when possible to avoid deep nesting.

Code Reference:

```
def area(width, height):
    return width * height

def print_area(w, h):
    print(area(w, h))

print_area(4, 5) # 20
```

*What it shows: area computes and returns; print_area reuses area instead of duplicating $w * h$.*

Topic guide

What it actually is

A function is a named, reusable mini-program inside your program. You call it by name with arguments; it may return a result.

When to use it

Use a function when the same logic appears twice, when a task has a clear name, or when you want to test one piece separately.

Where to use it

Calculations, formatting output, validating input, API handlers, and splitting large scripts into readable pieces.

Real use example

A password checker function `validate(pw)` returns `True/False` and is reused on signup, login, and reset forms.

Chapter 6: Object-Oriented Programming

Chapter focus: Object-Oriented Programming

Object-oriented programming models real things as objects with data (attributes) and behavior (methods). A class is the blueprint; `__init__` sets up new objects. `self` refers to the current instance. Inheritance lets a child class reuse and extend a parent class. OOP helps when many objects share the same structure - users, products, bank accounts.

Code Reference:

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount

acc = BankAccount("Mia", 100)
acc.deposit(50)
print(acc.balance) # 150
```

What it shows: `__init__` sets owner and balance; deposit updates balance on the object acc.

Topic guide

What it actually is

OOP organizes code around objects - bundles of data and functions that operate on that data.

When to use it

Use classes when you have many similar entities, need clear structure in large programs, or model real-world things (Student, Order, Sensor).

Where to use it

Games (Player, Enemy), apps (User, Post), GUIs, and libraries like Django models.

Real use example

A school app defines class `Student` with name and grades; each student object stores its own data while share methods like `add_grade()` work the same for everyone.

Chapter 7: Working with Files

Chapter focus: Working with Files

Use `pathlib.Path` for modern path handling. Check `Path('data.csv').exists()` before reading. `Encoding='utf-8'` avoids character errors on Windows.

Code Reference:

```
with open("notes.txt", "w") as f:
    f.write("Line 1\nLine 2\n")

with open("notes.txt") as f:
    print(f.read())
```

What it shows: First block writes two lines; second reads the whole file back.

Topic guide

What it actually is

File handling is reading from or writing to disk. It connects your program's memory to persistent storage.

When to use it

Use files to save logs, load configs, export reports, read CSV datasets, or store user progress.

Where to use it

Data pipelines, backup scripts, game save files, reading homework datasets, generating invoices.

Real use example

A backup script copies every `.docx` from `~/Documents` to `~/Backup` using `shutil` and `Path.glob('*.*docx')`.

Chapter 8: How Coding Works and Python Libraries

Chapter focus: How Coding Works and Python Libraries

Data science uses programming to collect, clean, analyze, and explain data. The typical flow: ask a question, load data (CSV, database), explore with summaries and charts, find patterns, and share results. Python is popular because libraries like NumPy, Pandas, and Matplotlib handle heavy work. You still need clean data and clear questions - tools alone do not guarantee good insights.

Code Reference:

```
import pandas as pd
df = pd.DataFrame({"sales": [120, 150, 90]})
print(df["sales"].mean())
print(df.describe())
```

What it shows: Pandas loads a table; mean() averages sales; describe() shows count, min, max, and more.

Topic guide

What it actually is

Data science is turning raw data into useful answers using statistics, visualization, and sometimes machine learning.

When to use it

When you have data and need trends, comparisons, forecasts, or evidence for decisions.

Where to use it

Marketing analytics, healthcare research, sports stats, finance, and school science projects.

Real use example

A store loads monthly sales CSV, computes average per product, plots a bar chart, and finds which item dropped 20% - guiding the next order.

Chapter 9: Getting Programs to Work

Chapter focus: Getting Programs to Work

A project combines many skills into one working program: input, logic, data structures, maybe files or libraries. Plan small - one clear goal, a few features, then test. Break the project into functions or steps. Debugging is normal. Finishing a project teaches more than reading ten chapters because you see how pieces connect.

Code Reference:

```
# Mini project: guess the number
import random
secret = random.randint(1, 10)
for _ in range(3):
    g = int(input("Guess 1-10: "))
    if g == secret:
        print("Correct!"); break
else:
    print("Out of tries. Answer:", secret)
```

What it shows: Combines random, input, loop, conditionals, and break in one playable game.

Topic guide**What it actually is**

A project is a complete program that solves a small real problem, not just a single concept demo.

When to use it

After learning basics - consolidate skills, portfolio pieces, homework capstones.

Where to use it

Courses, hackathons, personal GitHub, job interviews (show working code).

Real use example

A student builds a contact book CLI that saves names and phones to a JSON file - uses dicts, files, functions, and loops in one app.